Module Cloud Computing & Internet des Objets — Licence 3 Informatique

Conception et déploiement d'une solution IoT

Surveillance intelligente d'un laboratoire informatique

ESP32 · Node.js · MongoDB · Cloud

RÉALISÉ PAR

El Hadji Fallou Bousso

Moustapha Diop

Abdou Aziz Diop

Modou Dienne

Année universitaire : 2025 – 2026

Dépôt GitHub : github.com/serignefalloubousso99-dot/Projet-Cloud-IOT

Sommaire

1 Contexte et problématique
2 Objectifs et cahier des charges
3 Analyse du besoin et choix techniques
4 Architecture générale de la solution
5 Schéma de câblage (ESP32)
6 Modèle de données (MongoDB)
7 Fonctionnement : gestion du seuil et des alertes
8 L'API Node.js
9 Le tableau de bord Web
10 Sécurité
11 Déploiement dans le Cloud
12 Tests et validation
13 Difficultés rencontrées et solutions
14 Conclusion et perspectives
15 Annexes (livrables et liens)

1 Contexte et problématique

Les laboratoires informatiques regroupent de nombreux équipements (ordinateurs, serveurs, matériel réseau) qui dégagent une quantité importante de chaleur en fonctionnement. En cas de panne de la climatisation ou de début de surchauffe électrique, la température peut rapidement dépasser un seuil critique et endommager le matériel.

Problématique : comment concevoir et déployer une solution IoT capable de surveiller en continu l'environnement d'un laboratoire, de détecter automatiquement l'atteinte d'un seuil, de déclencher une alerte locale et distante, d'enregistrer les données dans le Cloud et de présenter des historiques selon plusieurs périodes ?

2 Objectifs et cahier des charges

L'objectif est de réaliser une solution IoT complète permettant de collecter, transmettre, stocker, analyser et visualiser des données environnementales en temps réel à partir d'une carte ESP32.

Les fonctionnalités obligatoires retenues sont :

- Lire les données d'un capteur et les afficher sur le moniteur série.
- Configurer un seuil et détecter automatiquement son dépassement.
- Activer / arrêter automatiquement une alerte sonore (buzzer) et visuelle (LED).
- Transmettre les mesures et l'état d'alerte à une API Node.js.
- Enregistrer les données dans MongoDB.
- Afficher un tableau de bord distant avec les historiques (matin, soir, nuit, hebdomadaire).
- Déployer l'ensemble de la solution dans le Cloud.

3 Analyse du besoin et choix techniques

3.1 Grandeur surveillée et capteur

La grandeur physique surveillée est la **température** (avec l'humidité en complément). Le capteur retenu est le **DHT11**, un capteur numérique de température et d'humidité, standard, stable et adapté aux projets IoT éducatifs sur ESP32. Contrairement à un capteur purement analogique, il fournit directement des valeurs numériques exploitables et une seconde grandeur (l'humidité) qui enrichit le suivi.

3.2 Justification du seuil

Une salle informatique en fonctionnement normal se situe autour de 20–25 °C. Nous fixons le seuil critique à **30 °C** : au-delà, on considère qu'il existe une surchauffe anormale (climatisation en panne ou échauffement électrique) mettant en danger le matériel. Afin d'éviter que l'alerte ne « clignote » lorsque la valeur oscille autour du seuil, une **hystérésis de 2 °C** est appliquée : l'alerte se déclenche à 30 °C mais ne

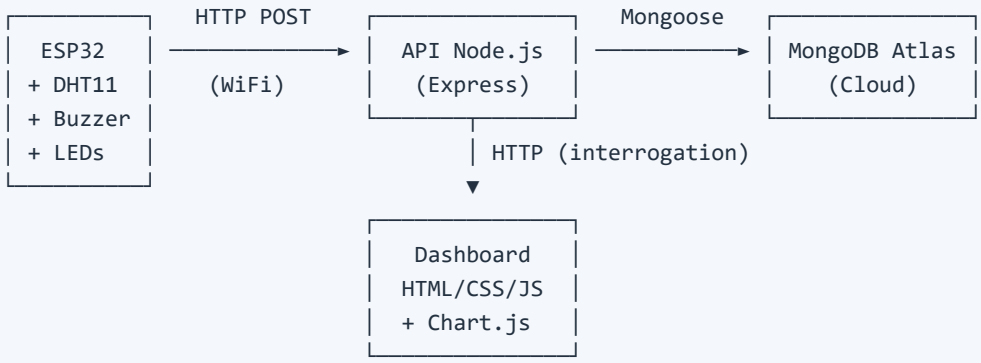
s'arrête qu'en dessous de 28 °C. Le seuil est **configurable** et doit être adapté à la température ambiante de la salle.

3.3 Pile technologique

Couche	Technologies
Embarqué	ESP32, C/C++ (Arduino IDE), DHT sensor library (Adafruit), WiFi.h, HTTPClient.h
Backend (API)	Node.js, Express.js, Mongoose, dotenv, cors
Base de données	MongoDB Atlas (Cloud)
Frontend	HTML5, CSS3, JavaScript (Vanilla), Chart.js
Déploiement	Render (API + dashboard), MongoDB Atlas
Outils	Git / GitHub, Postman

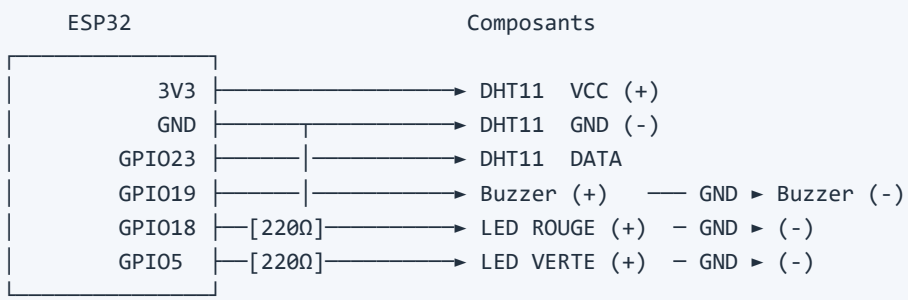
4 Architecture générale de la solution

La solution suit une chaîne complète de bout en bout : le capteur mesure, l'ESP32 transmet en WiFi via HTTP, l'API traite et stocke dans MongoDB, et le tableau de bord consulte l'API pour afficher les données.



> L'API constitue le cœur du système : elle est la **source de vérité** pour l'état d'alerte, ce qui garantit la cohérence des données quel que soit le client (ESP32 ou tableau de bord).

5 Schéma de câblage (ESP32)



Composant	Broche ESP32	Remarque
DHT11 — VCC	3V3	Le DHT11 est stable en 3.3 V
DHT11 — DATA	GPIO 23	Résistance pull-up 10 kΩ (souvent intégrée au module)
Buzzer (+)	GPIO 19	Buzzer actif
LED rouge (alerte)	GPIO 18	+ résistance 220 Ω
LED verte (normal)	GPIO 5	+ résistance 220 Ω

Les broches ont été choisies pour éviter les « strapping pins » (comme GPIO 12) qui peuvent perturber le démarrage de l'ESP32.

6 Modèle de données (MongoDB)

Deux collections sont utilisées, ce qui rend le calcul des historiques simple et efficace.

6.1 Collection `measurements` — toutes les mesures

Champ	Type	Rôle
deviceId	texte	Identifiant de la salle (ex. LABO-INFO-01)
temperature	nombre	Température mesurée (°C)
humidity	nombre	Humidité (%)
threshold	nombre	Seuil au moment de la mesure
state	texte	NORMAL ou ALERT
timestamp	date	Horodatage de la mesure

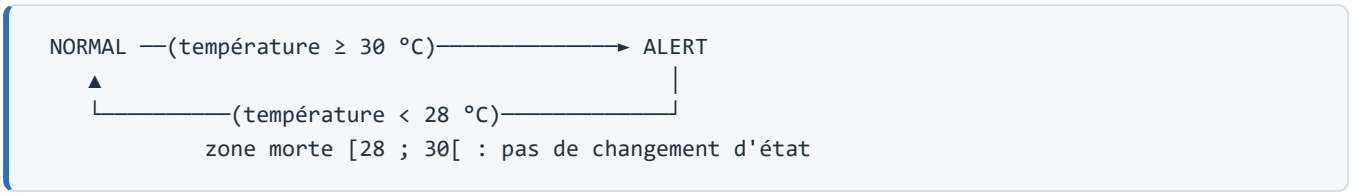
6.2 Collection `alertEvents` — les épisodes d'alerte

Champ	Type	Rôle
deviceId	texte	Salle concernée
startTime	date	Heure de début de l'alerte
endTime	date	Heure de fin (null si en cours)
thresholdAtEvent	nombre	Seuil déclencheur
triggerValue	nombre	Température ayant déclenché l'alerte
maxValue	nombre	Pic de température de l'épisode
status	texte	ONGOING ou RESOLVED

> Un **épisode d'alerte** correspond à un seul document dans `alertEvents`, créé au dépassement et clôturé au retour à la normale. C'est ce qui répond à l'exigence « éviter plusieurs alertes identiques » et permet de compter facilement les alertes par jour.

7 Fonctionnement : gestion du seuil et des alertes

Le système repose sur une **machine à états** à deux états (NORMAL et ALERT) avec hystérésis, implémentée à la fois sur l'ESP32 (pour l'alerte locale immédiate) et dans l'API (source de vérité pour l'enregistrement).



7.1 Au déclenchement (température ≥ seuil)

- Le buzzer et la LED rouge s'activent (alerte locale).
- L'état « ALERT » est transmis à l'API.
- Un évènement d'alerte est créé (heure de début).
- Le tableau de bord passe au rouge.

7.2 Au retour à la normale (température < seuil − hystérésis)

- Le buzzer s'arrête, la LED verte se rallume.
- L'évènement d'alerte est clôturé (heure de fin).
- Le tableau de bord repasse au vert.

Tant que l'état ne change pas, **aucune alerte supplémentaire n'est générée** : seul le pic de température de l'épisode est mis à jour.

8 L'API Node.js

L'API expose les points d'entrée suivants (les écritures sont protégées par une clé API) :

Méthode	Route	Description
POST 🔒	/api/measures	Enregistrer une nouvelle mesure
GET	/api/measures	Lister les mesures (filtres date, période, état)
GET	/api/measures/latest	Dernière mesure + état courant
GET	/api/alerts	Liste des évènements d'alerte
GET	/api/alerts/active	Alerte(s) en cours
GET	/api/alerts/resolved	Retours à l'état normal
GET	/api/history/daily	Historique journalier : matin / soir / nuit
GET	/api/history/weekly	Historique hebdomadaire
GET	/api/history/monthly	Historique mensuel

Les réponses sont normalisées au format JSON (`{ success, data, meta }`) et les codes d'erreur adaptés sont retournés : 201 (créé), 400 (requête invalide), 401 / 403 (authentification), 404 (introuvable), 500 (erreur serveur).

Plages horaires retenues

Période	Plage horaire
Matin	06h00 – 12h00
Soir	12h00 – 20h00
Nuit	20h00 – 06h00

9 Le tableau de bord Web

Le tableau de bord affiche en temps réel la dernière mesure, le seuil, l'état, les **compteurs d'alertes** (du jour, de la semaine et au total), ainsi que les historiques. Il distingue clairement cinq situations : **état normal** (vert), **état d'alerte** (rouge), **signal perdu** (mesure trop ancienne), **aucune donnée** et **perte de connexion** à l'API. L'interface est **responsive** : sur mobile, une barre de navigation fixe en bas d'écran permet d'atteindre chaque section.

Surveillance du laboratoire informatique

Salle LABO-INFO-01

En ligne

Normal

Température 25°C, sous le seuil de 30°C.

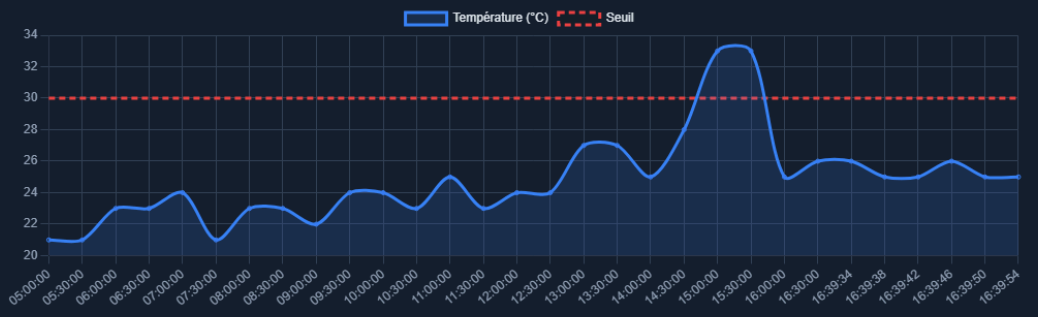
25 °C
TEMPÉRATURE ACTUELLE

HUMIDITÉ	SEUIL CONFIGURÉ	DERNIÈRE MESURE
52 %	30 °C	04/07/2026 16:39:54

ALERTES AUJOURD'HUI	ALERTES CETTE SEMAINE	TOTAL DES ALERTES
1	5	6

Température en temps réel

La ligne pointillée représente le seuil



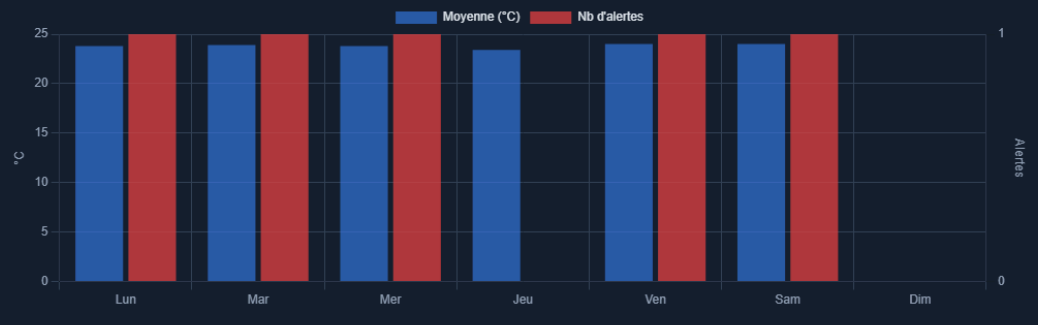
Historique du jour

jj/mm/aaaa

Matin (06h00 – 12h00)		Soir (12h00 – 20h00)		Nuit (20h00 – 06h00)	
Mesures	12	Mesures	16	Mesures	12
Moyenne	23.2°C	Moyenne	26.5°C	Moyenne	21.3°C
Min / Max	21 / 25°C	Min / Max	24 / 33°C	Min / Max	20 / 23°C
Alertes	0	Alertes	1	Alertes	0

Historique hebdomadaire

jj/mm/aaaa



Dernières alertes

DÉBUT	FIN	PIC (°C)	SEUIL	DURÉE	STATUT
04/07/2026 15:00:00	04/07/2026 16:00:00	33	30	60 min 0s	Résolue
03/07/2026 15:00:00	03/07/2026 16:30:00	34	30	90 min 0s	Résolue
01/07/2026 15:30:00	01/07/2026 16:00:00	33	30	30 min 0s	Résolue



Figure 1 — Tableau de bord en état normal (bandeau et liseré verts) : température, humidité, seuil, compteurs d'alertes et historiques.

Surveillance du laboratoire informatique

Salle LABO-INFO-01

En ligne

ALERTE — Surchauffe

Température 33°C, seuil 30°C dépassé (pic 33°C).

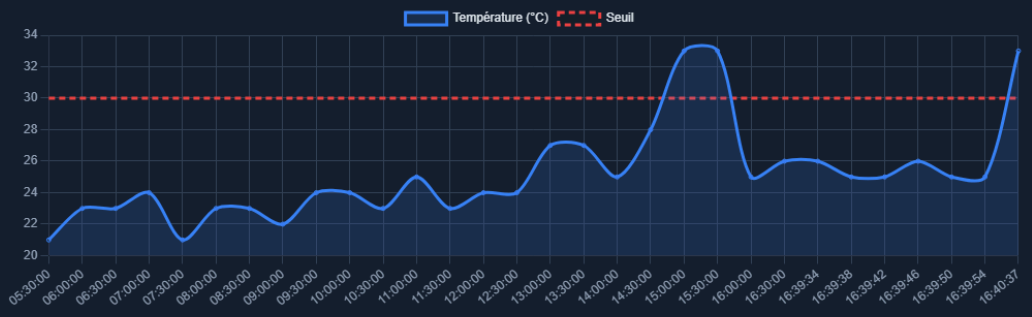
33 °C
TEMPÉRATURE ACTUELLE

HUMIDITÉ	SEUIL CONFIGURÉ	DERNIÈRE MESURE
62 %	30 °C	04/07/2026 16:40:37

ALERTES AUJOURD'HUI	ALERTES CETTE SEMAINE	TOTAL DES ALERTES
2	6	7

Température en temps réel

La ligne pointillée représente le seuil



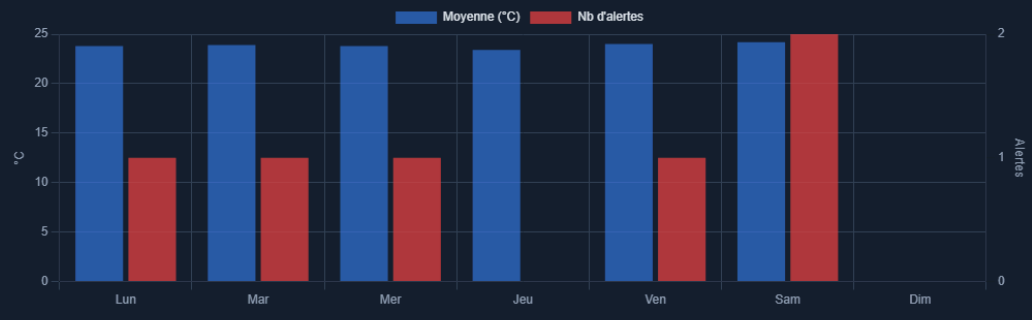
Historique du jour

jj/mm/aaaa

Matin (06h00 – 12h00)		Soir (12h00 – 20h00)		Nuit (20h00 – 06h00)	
Mesures	12	Mesures	17	Mesures	12
Moyenne	23.2°C	Moyenne	26.9°C	Moyenne	21.3°C
Min / Max	21 / 25°C	Min / Max	24 / 33°C	Min / Max	20 / 23°C
Alertes	0	Alertes	2	Alertes	0

Historique hebdomadaire

jj/mm/aaaa



Dernières alertes

DÉBUT	FIN	PIC (°C)	SEUIL	DURÉE	STATUT
04/07/2026 16:40:37	—	33	30	—	En cours
04/07/2026 15:00:00	04/07/2026 16:00:00	33	30	60 min 0s	Résolue
03/07/2026 15:00:00	03/07/2026 16:30:00	34	30	90 min 0s	Résolue



Figure 2 — Tableau de bord en état d'alerte : surchauffe à 33 °C, pic dépassant la ligne de seuil, compteurs à jour et alerte « en cours ».

10 Sécurité

- **Protection de l'API :** les écritures (POST des mesures) exigent une clé secrète transmise dans l'en-tête HTTP `x-api-key`. Sans clé valide, la requête est rejetée (401 / 403).
- **Variables d'environnement :** toutes les informations sensibles (chaîne de connexion MongoDB, clé API) sont stockées dans un fichier `.env` (jamais versionné) et dans les variables d'environnement de la plateforme d'hébergement. Seul un modèle `.env.example` est publié.
- **Gestion des erreurs :** reconnexion WiFi automatique côté ESP32, vérification des lectures capteur, gestion centralisée des erreurs côté API, et détection de la perte de connexion côté tableau de bord.

11 Déploiement dans le Cloud

L'ensemble de la solution est déployé sur des services gratuits :

Élément	Plateforme	URL
Base de données	MongoDB Atlas	cluster0.susqbgt.mongodb.net / base labo_iot
API	Render (Web Service)	https://projet-cloud-iot.onrender.com
Tableau de bord	Render (Static Site)	https://projet-iot-dashboard.onrender.com

Le code est versionné sur GitHub ; chaque `push` déclenche automatiquement un redéploiement sur Render (intégration continue).

12 Tests et validation

La solution a été validée à plusieurs niveaux :

- **Tests de la logique métier** (machine à états et hystérésis) : vérification qu'un seul évènement d'alerte est créé par épisode, que le retour à la normale clôture correctement l'évènement, et que le pic de température est bien enregistré.

- **Tests de l'API de bout en bout** : authentification (401 / 403), validation des données (400), enregistrement, consultation, alertes et historiques — l'ensemble des routes a été vérifié.
- **Tests via Postman** : une collection est fournie pour rejouer manuellement chaque route (livrable).
- **Test de la chaîne complète** : l'ESP32 réel envoie ses mesures à l'API déployée et celles-ci s'affichent en direct sur le tableau de bord public.

13 Difficultés rencontrées et solutions

Difficulté	Solution apportée
Éviter les alertes répétées autour du seuil	Machine à états avec hystérésis de 2 °C (déclenchement à 30 °C, arrêt à 28 °C)
Cohérence de l'état entre l'ESP32 et la base	L'API recalcule l'état à partir du précédent : elle est la source de vérité
Pilote USB (CP2102) de l'ESP32 manquant sous Windows	Installation du pilote CP210x de Silicon Labs, puis détection correcte du port série
Communication HTTPS de l'ESP32 vers l'API déployée	Utilisation de WiFiClientSecure pour l'envoi sécurisé des mesures
Seuil non adapté à une salle déjà chaude (l'alerte ne se coupait pas)	Calibration du seuil au-dessus de la température ambiante (seuil configurable)
Mise en veille de l'hébergement gratuit (Render)	Connaissance du délai de réveil et « ping » keep-alive périodique

14 Conclusion et perspectives

Ce projet a permis de concevoir, développer et déployer une solution IoT complète et fonctionnelle, couvrant l'ensemble de la chaîne : acquisition par capteur, traitement embarqué, transmission réseau, stockage dans le Cloud, et visualisation à distance. Les exigences du cahier des charges (seuil configurable, alerte locale et distante, anti-répétition, historiques, déploiement Cloud, sécurité minimale) sont satisfaites, et la chaîne a été démontrée de bout en bout avec le matériel réel.

Plusieurs améliorations pourraient être envisagées :

- Ajout de plusieurs capteurs / salles (le modèle est déjà extensible via `deviceId`).
- Notifications push ou e-mail lors d'une alerte.
- Communication temps réel par WebSocket (Socket.IO) au lieu de l'interrogation périodique.
- Authentification par comptes utilisateurs pour l'accès au tableau de bord.

15 Annexes — livrables et liens

Livrable	Emplacement
Code source ESP32	dossier `esp32/`
Code source API Node.js	dossier `api/`

Modèle de données MongoDB	`api/models/`
Tableau de bord Web	dossier `dashboard/`
Fichier .env.example	`api/.env.example`
Schéma de câblage	`esp32/README.md`
Collection Postman	`docs/Postman_Collection.json`
Guide de déploiement	`docs/DEPLOIEMENT.md`
Lien de l'API déployée	https://projet-cloud-iot.onrender.com
Lien du tableau de bord	https://projet-iot-dashboard.onrender.com
Dépôt GitHub	github.com/serignefalloubousso99-dot/Projet-Cloud-IOT